
中国矿业大学计算机学院

2023 级本科生课程报告

课程名称： 数据挖掘基础

报告题目： 特征预处理方法及应用

报告时间： 2025.11.21

学生姓名： 王志豪

学 号： 08232337

专 业： 数据科学与大数据技术

任课教师： 祝汉城

课程报告成绩评定及评分依据

序号	评分依据	分值	实际得分
1	课程报告描述了数据挖掘技术或算法的基本原理	20	
2	课程报告详细描述了待解决的问题，如有可能参考教材给出解决问题的过程	30	
3	课程报告使用 Python 程序设计语言（附源代码）对待解决问题进行了实验，给出了相关实验结果或可视化结果	40	
4	课程报告给出了个人学习体会	10	
总分			
教师评语： 教师签名： 年 月 日			

目 录

1 绪论	1
2 问题描述	1
3 特征预处理方法原理	1
3.1 模型架构	1
3.2 模块设计原理与分析	2
3.2.1 类别特征数字化 (Label Encoding)	2
3.2.2 构造特征 (Feature Engineering)	3
3.2.3 独热编码处理 (One-Hot Encoding)	3
3.2.4 逻辑回归 (Logistic Regression)	4
4 源代码及实现结果	4
6 个人体会	7

1. 绪论

员工离职预测是人力资源管理中的关键课题，其目标是借助历史数据与分析技术识别影响离职的主要因素，并在员工产生离职倾向前提供预警，以降低人才流失、提升组织稳定性并优化管理决策。随着企业数字化建设的推进，员工在年龄、岗位、满意度、绩效、加班情况、任职年限等多维度的数据被持续记录，使得构建基于数据驱动的预测模型成为可能。相比依赖经验的传统管理方式，机器学习能够系统地提取特征模式并输出量化的离职概率，从而实现更精确、可解释且可重复的风险识别。本项目构建了一个基于逻辑回归和特征工程的预测模型，通过数据预处理、类别特征编码、构造交叉特征以及独热编码后的高维稀疏表示，捕捉员工特征与离职行为之间的线性与交互关系，并利用分类模型输出离职概率。模型不仅用于预测未来员工的离职风险，还可通过权重分析揭示关键影响因素，为企业制定针对性的人才保留措施提供支持。总体而言，员工离职预测具有重要的理论与实践价值，是现代组织增强人才管理能力的重要技术手段。

2. 问题描述

本研究的主要问题在于通过对员工基本情况数据的挖掘，以对员工将来是否会离职进行预测。但是员工离职行为受多种因素共同作用，包括年龄、岗位特性、工作满意度、绩效表现、工作负荷、晋升通道、个人发展预期等，这些变量之间往往具有复杂的交互关系，单纯依赖管理经验难以获得准确判断。面对规模化组织中不断增长的数据量与维度，传统的人工分析方法已经无法满足预测的准确性与实时性需求。因此，本项目提出利用机器学习方法对员工离职进行建模与预测，希望通过对历史数据的系统分析，构建一个能够自动识别特征模式并输出预测结果的模型。综上所述，通俗来说，本研究所面对的问题其实是一个“二分类”问题，其主要分为两个部分，首先对数据进行预处理，包括数据的清洗、规约等等；而后建立特定的预测模型，将处理后的数据作为模型的输入，训练模型参数；最后将测试的数据作为模型的输入，以获取最终预测的结果。

3. 特征预处理方法原理

3.1 模型架构

由图 1 可知，由于任务比较直接，相对来说解决逻辑比较线性，因此模型的架构也相应比较线性。主要的设计模块分为两大模块：1. 原始数据的预处理模块；2. 模型预测

模块。具体模块细节如图 2 所示，黄色模块即为预处理模块，蓝色模块即为预测模块。

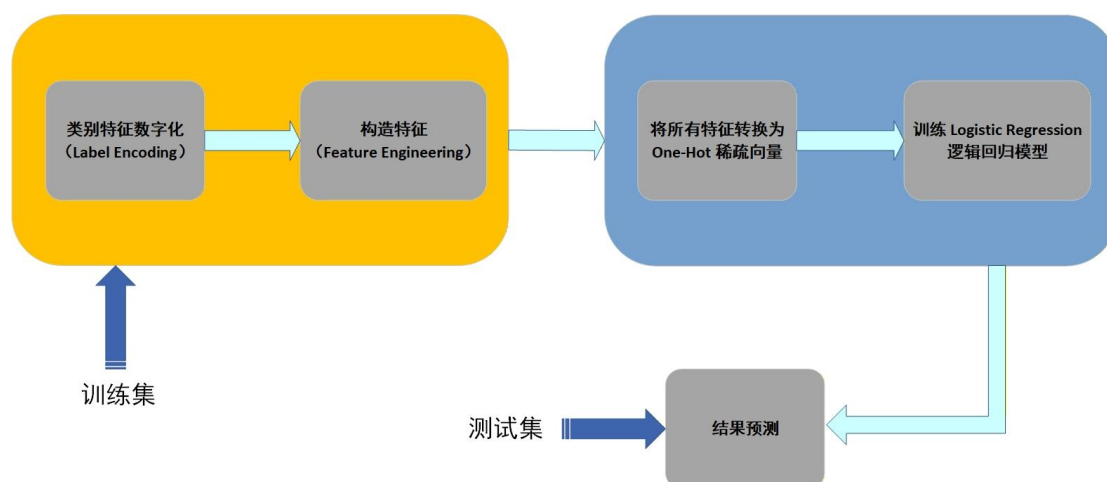


图 2 具体模块细节图

3.2 模块设计原理与分析

3.2.1 类别特征数字化 (Label Encoding)

经过初步的分析，我们可以从 train 和 test 中发现，所有的员工都已成年并且 StandHours 均为 80，这种所有人均一致的特征我们可以初步将其从数据集中剔除，因为可以初步判定这些值不会对员工是否离职产生影响。

在该模块，我们先将所有类别特征合并并建立字典。为什么呢？因为需要防止测试集中出现训练时未出现过的特征取值。众所周知，某些特征的取值并非数值，这样的特征在后续进行模型计算的时候是没法直接将类似于字符串式的特征代入模型的，因此我们需要将非数值特征转化为数值特征。

```
featArr=hstack((df_train[feat].values,df_test[feat].values))
feat_dict=make_feat_dict(featArr)
```

图 3 合并与数值转化

由图 3 可知，我们将训练集和测试集的每个特征的取值进行合并，然后进行 make_feat_dict 操作，make_feat_dict 的作用是：1. 找出所有唯一类别；2. 为每个类别分配一个整数 ID；3. 形成映射表。例如，department 特征中假如取值有三个: Sales, Research, HR。最后的映射表会是这样的：{'Sales':1, 'Research':2, 'HR':3}，但是这样还不够，我们还需要将 key 直接用对应的 value 进行替换，方法如图 4。

```
new_train=map_featid(feat,feat_dict,df_train[feat].values)
new_test=map_featid(feat,feat_dict,df_test[feat].values)
```

图 4 原非数值特征替代

这样每一个非数值的特征取值就由数值进行替代了，以便后续计算需要。

3.2.2 构造特征 (Feature Engineering)

该模块中，我先对年龄分为 5 岁一个区间，以提高模型对于非线性的处理能力。而后构造 7 类交叉特征，以将两个离散变量组合成一个更细粒度的标识符，进而让逻辑回归可以学习两个特征之间的交互影响。

AgeDistance	Age*100 + DistanceFromHome	年龄 × 距离（居住与年龄的耦合）
AgeEnvir	Age*10 + EnvironmentSatisfaction	年龄 × 环境满意度
JobRoleLevel	JobRole*10 + JobLevel	职位 × 职级
OverPer	OverTime*10 + PerformanceRating	加班 × 绩效
InvolvementPer	JobInvolvement*10 + PerformanceRating	工作投入 × 绩效
StockYear	StockOptionLevel*10 + YearsAtCompany	期权等级 × 司龄

图 5 七类交叉特征

其中交叉特征构造对象的选取则是按照实际生产中的经验来预先判断的，即先选在业务上看起来可能交互影响离职的对，然后在后续的验证过程中不断优化选取策略和对象。经过重新构造交叉特征后，我们将得到新的训练集和测试集。

3.2.3 独热编码处理 (One-Hot Encoding)

该模块中，我们首先将训练集和测试集做联合 fit，以保证测试集中不会出现未知类别。而后，我们将所有特征转换为 One-Hot 稀疏向量，经由以下操作，可以得到一个巨大、高维、稀疏的特征矩阵。

```
x_train=EncoderList[i].transform(df_train[feat].values.reshape(-1,1))
x_test=EncoderList[i].transform(df_test[feat].values.reshape(-1,1))
if i==0:
    X_train=x_train
    X_test=x_test
else:
    X_train=sparse.hstack((X_train,x_train))
    X_test=sparse.hstack((X_test,x_test))
```

图 6 独热编码的源代码

独热编码的原理就是将离散类别变量转换为“稀疏二元向量”，如果一个变量有 k 个可能取值，独热编码会将其映射为一个 k 维向量，其中：只有 1 个位置是 1，其他位

置全部是 0。例如：假设“部门 (Department)”有 3 类 (Sales, HR, R&D)，那么独热编码后就会变成：Sales (1, 0, 0)，HR (0, 1, 0)，R&D (0, 0, 1)，这样就能避免数字间大小关系颠覆类别间的平级关系了。

3.2.4 逻辑回归 (Logistic Regression)

我们直接调用逻辑回归的函数，进行模型的训练。

```
clf=LogisticRegression()  
clf.fit(X_train,y_train)
```

图 7 逻辑回归模型训练

经由训练好的逻辑回归模型进行测试，我们可以直接得到最终的 test 结果 submission。具体的逻辑回归模型可以简单的描述为两个过程。首先我们将独热编码后得到的高维稀疏矩阵 X (形状为 $n \times d$ ， n =样本数， d =特征维数) 带入线性方程中进行计算：

$$z^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + b$$
$$p^{(i)} = \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$$

图 8 逻辑回归模型计算公式

而后再将计算所得带入 sigmoid 函数中进行压缩，以压缩到 (0, 1) 概率区间中。这里我们设定阈值 0.5，我们认为大于 0.5 的则是会离职，小于 0.5 的则是不会离职。

```
y_test=clf.predict(X_test)  
submissionFile=pd.DataFrame({'result':y_test})  
submissionFile.to_csv("submission.csv",index=False)
```

图 9 逻辑回归模型测试

4. 源代码及实现结果

code_string.py：数据预处理，生成新训练集和测试集

```

from numpy import *
import pandas as pd
from scipy import sparse
from sklearn.preprocessing import OneHotEncoder
from scipy.sparse import csr_matrix
from sklearn.linear_model import LogisticRegression
import os

def load_data():
    """
    读取原始 CSV 数据并返回 DataFrame

    返回:
    - df_train: 原始训练数据 (未处理)
    - df_test: 原始测试数据 (未处理)
    """
    df_train = pd.read_csv('data\\ori_train.csv')
    df_test = pd.read_csv('data\\ori_test.csv')
    return df_train, df_test

def make_feat_dict(featArr):
    """
    将一个特征值序列映射到连续的整数 id (从 1 开始)

    参数:
    - featArr: 可迭代的特征值序列 (通常是 train+test 拼接后的列)

    返回:
    - feat_dict: dict, {原始值: id}
    """
    feat_dict = {}; counter = 1
    for item in featArr:
        if (item not in feat_dict):
            feat_dict[item] = counter
            counter += 1
    return feat_dict

def map_featid(feat, feat_dict, featArr):
    """
    使用给定的映射字典将一系列原始特征值转换为 id 列, 并以 DataFrame 返回 (列名为 feat)

    参数:
    - feat: 字符串, 列名
    - feat_dict: {原始值: id}
    - featArr: 要转换的原始值序列 (通常为原始 DataFrame 中的列)

    返回:
    - pandas.DataFrame: 单列 DataFrame, 列名为 feat, 值为对应 id
    """
    feat_list = []
    for item in featArr:
        feat_list.append(feat_dict[item])
    return pd.DataFrame({feat: feat_list})

if __name__ == '__main__':
    # 主流程: 读取数据 -> 对指定类别特征建立 id 映射 -> 构造派生特征 -> 保存
    df_train, df_test = load_data()

# 需要统一映射 id 的类别特征 (保证 train/test 使用相同的 id)
feats = ["BusinessTravel", "Department", "EducationField", "Gender", "JobRole", "MaritalStatus", "Over18", "OverTime"]
for feat in feats:
    # 将训练集与测试集对应列拼接在一起, 保证所有可能取值都被包含进映射
    featArr = hstack((df_train[feat].values, df_test[feat].values))
    feat_dict = make_feat_dict(featArr)
    # 将原始值替换为映射 id (以 DataFrame 形式替换原列)
    new_train = map_featid(feat, feat_dict, df_train[feat].values)
    new_test = map_featid(feat, feat_dict, df_test[feat].values)
    del df_train[feat]
    del df_test[feat]
    df_train[feat] = new_train
    df_test[feat] = new_test

# 构造若干派生特征 (与原样本相同的业务逻辑)
# Age 分桶 (将年龄按 5 岁一个区间)
df_train['Age'] = pd.DataFrame({'Age': df_train['Age'].values // 5})

# AgeDistance: 用年龄与距离组合成新的 ID (年龄*100 + 距离)
df_train['AgeDistance'] = pd.DataFrame({'AgeDistance': df_train['Age'].values * 100 + df_train['DistanceFromHome'].values})
df_test['AgeDistance'] = pd.DataFrame({'AgeDistance': df_test['Age'].values * 100 + df_test['DistanceFromHome'].values})

# AgeEnvir: 年龄与环境满意度组合
df_train['AgeEnvir'] = pd.DataFrame({'AgeEnvir': df_train['Age'].values * 10 + df_train['EnvironmentSatisfaction'].values})
df_test['AgeEnvir'] = pd.DataFrame({'AgeEnvir': df_test['Age'].values * 10 + df_test['EnvironmentSatisfaction'].values})

# JobRoleLevel: 职位角色与职级组合
df_train['JobRoleLevel'] = pd.DataFrame({'JobRoleLevel': df_train['JobRole'].values * 10 + df_train['JobLevel'].values})
df_test['JobRoleLevel'] = pd.DataFrame({'JobRoleLevel': df_test['JobRole'].values * 10 + df_test['JobLevel'].values})

# OverPer: 是否加班与绩效组合
df_train['OverPer'] = pd.DataFrame({'OverPer': df_train['OverTime'].values * 10 + df_train['PerformanceRating'].values})
df_test['OverPer'] = pd.DataFrame({'OverPer': df_test['OverTime'].values * 10 + df_test['PerformanceRating'].values})

# InvolvementPer: 工作投入度与绩效组合
df_train['InvolvementPer'] = pd.DataFrame({'InvolvementPer': df_train['JobInvolvement'].values * 10 + df_train['PerformanceRating'].values})
df_test['InvolvementPer'] = pd.DataFrame({'InvolvementPer': df_test['JobInvolvement'].values * 10 + df_test['PerformanceRating'].values})

# StockYear: 股票期权等级与公司年限组合
df_train['StockYear'] = pd.DataFrame({'StockYear': df_train['StockOptionLevel'].values * 10 + df_train['YearsAtCompany'].values})
df_test['StockYear'] = pd.DataFrame({'StockYear': df_test['StockOptionLevel'].values * 10 + df_test['YearsAtCompany'].values})

# 将处理后的数据保存, 供建模脚本使用
df_train.to_csv("data\\train.csv", index=False)
df_test.to_csv("data\\test.csv", index=False)

```

图 10 code_string.py 源代码

logistic_regression_model.py: 独热编码, 模型训练与结果预测

```
def load_data():
    """
    从预处理后文件中读取训练/测试数据

    返回:
    - df_train: 已经预处理并保存到 `data/train.csv` 的训练数据(DataFrame)
    - df_test: 已经预处理并保存到 `data/test.csv` 的测试数据(DataFrame)
    """
    df_train = pd.read_csv('data\\train.csv')
    df_test = pd.read_csv('data\\test.csv')
    return df_train, df_test

def initEncoderList(df_train, df_test):
    """
    为每个指定特征创建并 fit 一个 OneHotEncoder, 返回编码器列表。

    关键点:
    - 为保证 train/test 使用相同的编码, encoder 使用 `hstack((train, test))` 的值来 fit
    - 返回的 EncoderList 的顺序与 `onehot_encode` 中使用的特征顺序必须一致

    返回:
    - EncoderList: list[OneHotEncoder]
    """
    feats = [
        "Age", "BusinessTravel", "Department", "DistanceFromHome", "Education", "EducationField", "EmployeeNumber",
        "EnvironmentSatisfaction", "Gender", "JobInvolvement", "JobLevel", "JobRole", "JobSatisfaction", "MaritalStatus", "MonthlyIncome",
        "NumCompaniesWorked", "OverTime", "PercentSalaryHike", "PerformanceRating", "RelationshipSatisfaction", "StandardHours",
        "StockOptionLevel", "TotalWorkingYears", "TrainingTimesLastYear", "WorkLifeBalance", "YearsAtCompany", "YearsInCurrentRole",
        "YearsSinceLastPromotion", "YearsWithCurrManager", "AgeDistance", "AgeEnvir", "JobRoleLevel", "OverPer", "InvolvementPer", "StockYear"
    ]

    EncoderList = []
    for feat in feats:
        # 为每个特征创建编码器并用 train+test 的值 fit
        enc = OneHotEncoder()
        arr = hstack((df_train[feat].values, df_test[feat].values)).reshape(-1, 1)
        enc.fit(arr)
        EncoderList.append(enc)
    return EncoderList

def onehot_encode(df_train, df_test):
    """
    使用 pre-fit 的 EncoderList 对训练集与测试集逐列进行 one-hot 转换,
    并将所有列的 one-hot 矩阵水平拼接得到最终的稀疏特征矩阵用于训练与预测。

    流程:
    1. 调用 initEncoderList 获取已 fit 的编码器列表
    2. 对每个特征使用对应的编码器进行 transform, 得到稀疏矩阵
    3. 将得到的稀疏矩阵按列拼接成最终的 X_train / X_test
    4. 训练 LogisticRegression 并对 X_test 做预测, 输出 `submission.csv`
    """
    EncoderList = initEncoderList(df_train, df_test)
    # 目标变量
    y_train = df_train["Attrition"].values
    feats = ["Age", "BusinessTravel", "Department", "DistanceFromHome", "Education", "EducationField", "EmployeeNumber",
        "EnvironmentSatisfaction", "Gender", "JobInvolvement", "JobLevel", "JobRole", "JobSatisfaction", "MaritalStatus", "MonthlyIncome",
        "NumCompaniesWorked", "OverTime", "PercentSalaryHike", "PerformanceRating", "RelationshipSatisfaction", "StandardHours",
        "StockOptionLevel", "TotalWorkingYears", "TrainingTimesLastYear", "WorkLifeBalance", "YearsAtCompany", "YearsInCurrentRole",
        "YearsSinceLastPromotion", "YearsWithCurrManager", "AgeDistance", "AgeEnvir", "JobRoleLevel", "OverPer", "InvolvementPer", "StockYear"]
    # 逐列 transform 并水平拼接稀疏矩阵
```

```

for (i, feat) in enumerate(feats):
    x_train = EncoderList[i].transform(df_train[feat].values.reshape(-1,1))
    x_test = EncoderList[i].transform(df_test[feat].values.reshape(-1,1))
    if i == 0:
        X_train = x_train
        X_test = x_test
    else:
        X_train = sparse.hstack((X_train, x_train))
        X_test = sparse.hstack((X_test, x_test))
print("base data one hot over!")
print(shape(X_train), shape(X_test))

# 训练模型（此处使用默认参数的 LogisticRegression）
clf = LogisticRegression()
clf.fit(X_train, y_train)
# 如需保存模型可以使用 joblib.dump(clf, 'lr.model')
print("Train finish")

# 预测并保存结果到 submission.csv
y_test = clf.predict(X_test)
submissionFile = pd.DataFrame({'result': y_test})
submissionFile.to_csv("submission.csv", index=False)

if __name__ == '__main__':
    df_train, df_test = load_data()
    onehot_encode(df_train, df_test)

```

图 11 logistic_regression_model.py 源代码

Submission.csv: 预测结果文件，其中 1 表示离职，0 表示不离职。部分结果如下：

ID	result
1100	0
1101	0
1102	0
1103	0
1104	0
1105	0
1106	0
1107	0
1108	0
1109	0
1110	0
1111	0
1112	1
1113	0
1114	1

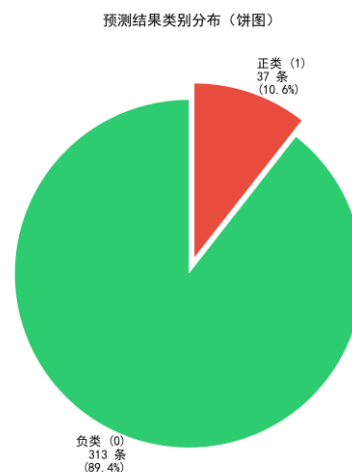


图 11 预测部分结果与汇总结果饼图

5. 个人体会

在本次员工离职预测项目中，我对数据处理流程、特征工程设计以及机器学习模型

训练有了更全面的认识。从项目一开始的原始数据探索，到后续的特征构造、编码转换，再到模型训练与结果分析，每一个环节都让我更加明确了数据质量和特征选择对于预测模型的重要性。在特征处理的过程中我发现本项目还有许多需要改进的地方，例如在选择对象进行交叉特征构建的时候过于随意，基本都是基于经验决策，更科学的方法是给训练集划分验证集，然后在验证集上计算各种评估指标（F1, Recall, Precision 等），再去判断选定的对象是否合适，还有就是最后进行二分类时，直接用 0.5 作为阈值也过于武断，也应该经过验证集的评估来动态调整阈值的大小。